

DigitalTwin.ai

Predictive Digital Twin for Mixed-Instrumentation Assembly Lines

Business Proposal & Working Prototype — Round 2 Submission

Scope

A reference solution design, phased rollout plan, and a working prototype demonstrating the core predictive mechanism (bottleneck forecasting, defect early-warning, and root-cause traceback) on a simulated 40-station mixed-model assembly line with realistically uneven sensor coverage.

Prepared for: Plant Operations & Engineering Leadership

Date: August 2026

Repository: <insert GitHub URL after publishing — see README.md>

All production data in this document and the accompanying prototype is simulated. Business-case figures are illustrative, derived from stated assumptions, and should be replaced with site-specific data before use in an investment decision.

Executive Summary

Most assembly lines pitched in digital-twin proposals are imaginary: uniformly instrumented, freshly built, single-site. Real lines are not. They are a patchwork built up over 10–25 years — a handful of modern robotic cells sitting next to PLCs that predate the plant's current ERP system, with a meaningful slice of stations still running on paper or tablet checklists because retrofitting them has never cleared the ROI bar on its own. This proposal is designed for that line, not the imaginary one.

We propose a digital twin that treats uneven instrumentation as the default condition to design for, not an edge case to work around later. The twin represents what it can measure directly (cycle time, vibration, torque, temperature at sensed stations) using transparent statistical process control, and infers what it cannot measure directly (defect risk at manual-checklist stations) from downstream confirmed outcomes and context factors like shift — without ever fabricating a sensor reading that isn't there. It is predictive rather than merely descriptive: it forecasts bottlenecks before they breach takt time, flags high-risk units before they reach final inspection, and ranks likely root-cause stations the moment a defect surfaces, even when that defect was introduced dozens of stations and thousands of units earlier.

Critically, every one of those claims is validated against ground truth in the accompanying prototype, not asserted. On a simulated 3,000-unit run with three injected, multi-causal fault modes, the prototype demonstrates:

- A bottleneck forecast for a wearing weld-arm bearing station with 517 units (~2.1 shifts) of lead time between the predictive alert and the point the station is actually, sustainably running over takt — enough warning to schedule a repair in the next maintenance window instead of reacting to an unplanned stoppage.
- Early-warning defect flags, raised before a unit reaches final inspection, that at a conservative operating point run ~85%+ precision (and up to 95% at the tightest threshold), so floor teams are not paged for noise — with a fully transparent precision/recall tradeoff curve for tuning that threshold as trust builds.
- Root-cause ranking that correctly identifies the true source station within its top-3 suspects 69% of the time — with the gap explicitly attributable to the ~30% of defects originating at a fully-manual station with no sensor trail at all, which is exactly the honest limitation this proposal's phased sensor-retrofit plan is designed to close.
- A working example of inferring elevated risk at a station with zero sensors, by statistically comparing confirmed-defect attribution rates across shifts — correctly and repeatedly flagging the night shift at a manual panel-fit inspection station as elevated risk, using only downstream outcomes.

The proposal that follows covers the problem in full, the solution design (modeling approach, predictive techniques, data-gap handling, integration approach, and scalability), the three stakeholder-specific views generated from one underlying model, a phased rollout roadmap that starts conservative and earns trust before expanding, a business case built on deliberately modest assumptions, and the key risks with concrete mitigations. The prototype referenced throughout is real, runnable code included in the accompanying repository — not a mockup.

1. Problem Framing

1.1 The line you actually have, not the line in the brochure

A typical mixed-model automotive assembly line has 30–50 stations spanning body construction, paint, and final assembly, built and re-equipped in phases over one to two decades. That history leaves three sensor tiers coexisting on the same line:

- Rich stations — modern robotic cells with multi-channel telemetry (cycle time, vibration, torque, temperature), usually installed in the last capital refresh.
- Basic stations — older PLCs exposing a single reliable signal, most often a cycle-time or part-present count, with no deeper diagnostic channel.
- Manual stations — a person with a checklist or a tablet. Common at fit-and-finish checks, trim inspections, and anywhere a human judgment call (does this look right?) hasn't yet been justified for automation.

Any twin that assumes uniform instrumentation either quietly ignores the manual and basic-tier stations (leaving real blind spots that undermine trust the first time a defect traces back to one of them) or demands a plant-wide sensor retrofit before it can deliver value (which rarely survives a capital approval process). Neither is acceptable. The twin has to be useful with the line as it exists on day one, and get incrementally better as instrumentation improves — not the other way around.

1.2 Why this is genuinely hard, not just an integration exercise

- Multi-causal, intermittent root causes. A defect is rarely one clean signal crossing one clean threshold. In our prototype's own fault design, an intermittent torque defect only manifests when a tool's own periodic drift coincides with an off-spec supplier part batch — neither factor alone is sufficient. Real lines have dozens of these interacting factors: equipment wear, operator variation, upstream part quality, and environmental conditions layer on top of each other.
- Defects surface late. A defect introduced at station 7 of 40 may not be caught until final inspection at station 40 — by which point hundreds of downstream units may carry the same undetected issue. Root-cause tracing after the fact means reconstructing a unit's entire path through the line, not just looking at the station that happened to fail the inspection.
- Live production is not a sandbox. PLCs and line-control logic run the plant. Modifying them carries real operational risk, and most plants only permit changes — including instrumentation retrofits — during scheduled, infrequent maintenance windows. A twin that requires write access to control logic, or requires unplanned downtime to install, is a non-starter regardless of its modeling sophistication.
- Different stakeholders need fundamentally different views of the same reality. A floor supervisor needs a real-time, in-the-moment signal they can act on in the next sixty seconds. A plant manager needs weekly trends to plan maintenance and staffing. Leadership needs a validated business case to approve further investment. Building three separate systems triples the integration risk and guarantees the numbers drift apart over time.
- Predictive claims must earn trust over time. A single high-profile false alarm — "the twin said this unit was defective and it wasn't" — can undo months of floor-level buy-in. The system has

to be validated continuously against real outcomes, not just at launch, and it has to be honest about its own uncertainty rather than presenting every output with false confidence.

- No two lines or sites are the same. Layout, equipment vintage, and sensor maturity all vary site to site. A solution hard-coded to one plant's specific PLC models and station layout does not generalize; the underlying pattern has to be portable even when the specific configuration isn't.

The design question this proposal answers

Not "how do we build the most sophisticated predictive model," but "how do we build something that is honest about what it knows and doesn't know, safe to integrate into a live line, and genuinely trusted by the people who have to act on its output every shift."

2. Solution Design

2.1 Design principles

- Work with the instrumentation that exists. Represent explicitly what's sensed; infer, don't fabricate, what isn't.
- Explainable over opaque. A floor supervisor needs to trust an alert in seconds. Transparent statistical methods (control charts, trend regression, cohort comparison) are favored over black-box models wherever they perform adequately — which, as the prototype shows, is most of the time.
- Predictive, not just descriptive. Every module is built to give a leading indicator with measurable lead time, not a dashboard that restates what already happened.
- Validated continuously. Every predictive claim is scored against confirmed outcomes on an ongoing basis, and thresholds are recalibrated based on that scoring, not set once and forgotten.
- Non-invasive integration. Read-only data taps, no write access to control logic, and all physical changes confined to scheduled maintenance windows.
- One model, three views. Floor, plant-manager, and leadership views are projections of the same underlying twin state, not three separately-maintained systems that can drift out of sync.

2.2 Reference architecture

The diagram below is the target architecture; the prototype implements every layer except live OT ingestion (which is replaced by the simulator described in Section 4, producing data in the same shape a real OPC-UA/MQTT tap would).

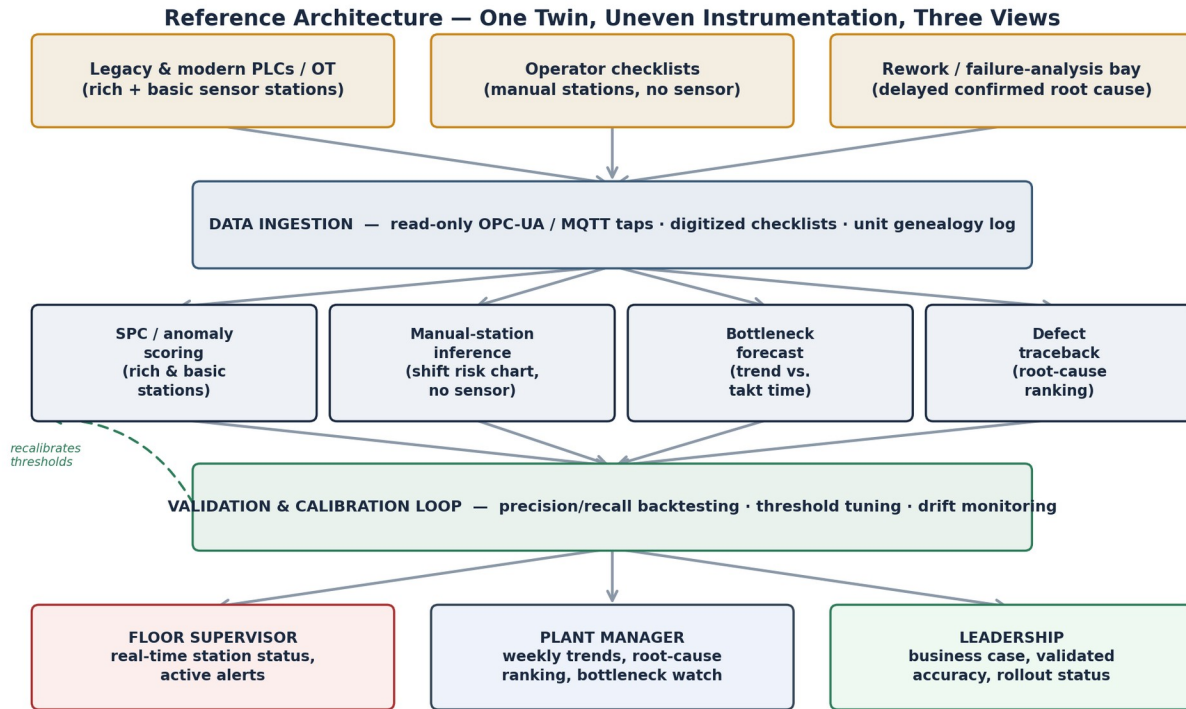


Figure 1. Reference architecture: ingestion is source-agnostic, the modeling layer runs four specialized components in parallel, a continuous validation loop recalibrates thresholds, and one twin state feeds three stakeholder-specific views.

2.3 Modeling approach — what's explicit vs. inferred

The single most important design decision in this proposal is treating sensor tier as a first-class concept the model reasons about, rather than a data-quality problem to clean up before modeling starts.

Station tier	What's represented explicitly	How gaps are handled	Example from the prototype
Rich (sensored)	Cycle time, vibration, torque, temperature as continuous channels	N/A — direct measurement	Torque channel directly captures an intermittent under-torque fault at the bolt-fastening station
Basic (single signal)	Cycle time (or equivalent single PLC counter)	Bottleneck forecasting relies on this one channel alone; other failure modes at this tier are invisible until corroborated downstream	A cycle-time-only trend is sufficient to forecast a takt-time breach with over 500 units of lead time

Station tier	What's represented explicitly	How gaps are handled	Example from the prototype
Manual (checklist only)	Operator pass/fail entry, shift, and (once confirmed) downstream defect attribution	No sensor reading is invented. Risk is inferred indirectly from patterns in confirmed outcomes, segmented by context (shift, operator, batch)	Night-shift risk at a panel-fit station is correctly and repeatedly flagged using only delayed, confirmed teardown attributions — zero direct sensor data

2.4 Predictive techniques, and how we validate them before trusting their output

Anomaly detection at sensed stations

We use an exponentially-weighted moving-average (EWMA) control chart on a trailing, causal window per station/channel — never looking ahead — rather than a heavier ML model. This is a deliberate tradeoff: mechanical wear and process drift are close to linear over the horizons that matter operationally (days, not months), and a control-chart z-score can be explained to a maintenance engineer in one sentence ("this reading has drifted outside its normal range"), whereas a neural network's anomaly score generally cannot. The prototype's SPC module correctly and cleanly surfaces the two independent injected faults (an intermittent torque dip and a gradual vibration/cycle-time drift from bearing wear) as the top anomaly sources on the line, with no manual tuning per fault.

Bottleneck forecasting

A trailing linear-regression trend on cycle time is compared against the station's takt time, and an alert fires only when the trend's slope is statistically significant (not just positive — this is what keeps single-cycle noise from generating false alarms) and extrapolates to a takt-time breach within a configurable lead horizon. In the prototype's wear-driven bottleneck case study, this produces a predictive alert 517 units (roughly two full shifts) before the station is actually, sustainedly running over takt — real lead time a maintenance team could use to schedule an intervention in the next planned window rather than responding to an unplanned stoppage.

Root-cause traceback for late-surfacing defects

Every unit carries a per-station "genealogy" of anomaly scores as it moves through the line — computed causally, in real time, as it passes each station, never reconstructed after the fact from future data. When a defect is caught at final inspection, the twin combines two views: the specific unit's own highest-anomaly stations (unit-level suspects), and a population-level statistical comparison of anomaly-score distributions between defective and non-defective units across the whole line (cohort-level contribution, using a standardized mean difference rather than a black-box feature-importance model). In the prototype, this correctly identifies the true injected fault station as the top cohort-level contributor by a wide margin, and lands the true source within the unit-level top-3 suspects 69% of the time overall.

Manual-station inference

Described fully in Section 2.5 below, since it is also the core answer to the data-gap question.

The validation loop

Nothing above is trusted at face value. The prototype includes a validation harness that scores every predictive claim against ground truth: precision/recall for early-warning defect flags (with a full threshold sweep showing the tradeoff), top-1/top-3 hit rate for root-cause ranking, and measured lead time for bottleneck alerts. In production, this same scoring runs continuously as a rolling backtest against confirmed outcomes (final-inspection results, rework-bay teardown findings), and feeds an explicit calibration policy: if precision at the currently-deployed threshold drops below an agreed floor, the threshold tightens automatically and supervisors are notified that alert sensitivity changed and why. This is what keeps the system honest over the life of the deployment, not just on day one.

2.5 Handling data gaps at sensor-poor stations

For manual, checklist-only stations, we do not attempt to infer a synthetic sensor reading — that would create false confidence exactly where the system is weakest. Instead, the twin treats the station's own checklist entries, combined with downstream confirmed root-cause attributions, as a slow, noisy but real signal, and runs an attribute control chart (a two-proportion z-test) over it, segmented by context factors most plants already log for free: shift, and in a fuller deployment, operator ID and part batch.

A key design detail: confirmed root-cause attributions in real plants come from a physical rework or failure-analysis process — a person tearing down a returned unit — which happens with a delay of hours to days, not instantly. The prototype explicitly models this confirmation lag, so the inference model never uses information it would not actually have yet in production. Despite that lag and despite having zero sensor readings at the station in question, the model correctly and consistently identifies an elevated-risk shift within roughly 800–900 units of accumulated data, purely from statistical patterns in confirmed downstream outcomes.

This inference approach is intentionally treated as a bridge, not a permanent solution — it is lower-confidence and slower to calibrate than direct sensing, which is precisely why the phased roadmap (Section 6) prioritizes low-cost retrofits at manual stations with the highest confirmed defect-attribution rate, scheduled into the plant's existing maintenance-window cadence rather than requiring a special downtime event:

- Fixed-position vision + lightweight edge model for checklist-style pass/fail judgments (e.g. a \$200–500 camera and a small on-device model checking panel gap or trim alignment), replacing subjective checks with a repeatable measurement.
- Low-cost vibration/current clamp IoT sensors (commodity units in the \$150–400 range) added to basic-tier stations during a scheduled window, upgrading them toward rich-tier coverage without a full PLC replacement.
- Barcode/RFID timestamp capture at manual stations, giving at least a reliable cycle-time signal even where a full checklist digitization isn't yet justified.

Each retrofit is prioritized by expected ROI using the very data the twin is already collecting: stations that repeatedly show up as high-confidence root-cause contributors, or where manual-station risk inference stays persistently elevated, move to the top of the retrofit list — the twin effectively directs its own instrumentation investment.

2.6 User experience — one twin, three views

All three views below are generated from the same underlying twin state (the same `dashboard_data.json` the prototype exports) — there is one source of truth, not three separately maintained reports that can silently drift apart.

View	Cadence	What it shows	Primary question it answers
Floor Supervisor	Real-time, this shift	A full line status strip across all 40 stations color-coded by state, active alerts with plain-language recommended actions, and a live shift defect-rate snapshot	What needs my attention in the next few minutes?
Plant Manager	Weekly / planning horizon	Defect-rate trend across shift blocks, station utilization vs. takt time, a ranked root-cause contributor table with confidence, and a bottleneck watchlist with forecasted breach timing	What should I schedule, staff, or escalate this week?
Leadership	Quarterly / investment horizon	The business case (cost avoided, payback period), the full validation summary (precision/recall, root-cause accuracy, measured lead time) so the numbers are auditable, and the rollout roadmap status	Is this working, and where should we invest next?

The accompanying prototype includes a working interactive dashboard (`dashboard/index.html`) implementing all three views against the simulated run's actual output — not mockup screenshots. See Section 4 and the repository README for how to open it.

2.7 Integration approach — working with live production, not around it

- Read-only by construction. Data leaves the OT network through standard, vendor-supported protocols (OPC-UA, MQTT) via a gateway that only subscribes to tag values — the twin has no write path back into PLC or line-control logic, ever. This removes an entire category of operational risk from the conversation with plant engineering.
- All physical changes confined to scheduled maintenance windows. New sensors, gateway hardware, or network taps are installed only during the plant's existing, infrequent maintenance windows — the rollout plan (Section 6) is explicitly sequenced around this constraint rather than assuming ad-hoc downtime.

- Edge buffering, not added load on legacy PLCs. An edge historian/buffer sits between the gateway and the modeling layer so that legacy equipment is never polled more aggressively than it already tolerates, and short network interruptions don't lose data.
- OT/IT network segmentation. The ingestion layer sits in a segmented zone consistent with standard OT security practice (e.g., a Purdue-model-style separation), with the modeling and presentation layers living entirely in IT infrastructure — no new inbound path is created into the control network.
- Manual-station digitization is additive, not disruptive. Checklist digitization (tablet entry, barcode scan) augments the existing manual process rather than replacing operator judgment, and is piloted at one station before wider rollout.

2.8 Scalability across lines, plants, and sites

The rich/basic/manual sensor-tier taxonomy used throughout this design is deliberately the unit of portability. Instead of hard-coding a specific station layout or PLC model, onboarding a new line or site means classifying its stations into this taxonomy and pointing the ingestion layer at its specific tag map — a configuration exercise, not a rewrite of the modeling layer.

- Config-driven onboarding. A new line's topology (station count, zone grouping, takt times, sensor tier per station) is defined in a configuration file analogous to this prototype's `line_config.py` — the SPC, inference, forecasting, and traceback modules run unmodified against any line described this way.
- Federated calibration. Each site's models are calibrated on that site's own data (equipment wear rates, defect patterns, and shift structures are genuinely site-specific), while sharing the same modeling framework and validation methodology — consistent, comparable metrics across a multi-site fleet without forcing a one-size-fits-all threshold.
- Marginal cost declines with each rollout. The first line carries the full cost of data-engineering and integration work; each additional line or site reuses the ingestion architecture, modeling code, and dashboard views, with cost concentrated in site-specific configuration and the sensor-tier gap-closing retrofits described in Section 2.5.
- A fleet-level view for multi-site leadership. As additional lines and sites onboard, the leadership view (Section 2.6) extends naturally into a portfolio rollout tracker — validated accuracy and business-case realization per site, informing where to invest next.

3. Target Users

Persona	Current pain	What the twin gives them
Floor Supervisor	Finds out about a bottleneck or defect trend only after it's already hurt the shift's output; no visibility into stations without a person physically checking them	A real-time station status strip and plain-language alerts with enough lead time to intervene, including at stations with no sensor of their own

Persona	Current pain	What the twin gives them
Plant Manager	Weekly planning relies on lagging inspection reports and tribal knowledge of "which station is always the problem"	A validated, ranked root-cause contributor list and a forward-looking bottleneck watchlist to inform maintenance and staffing decisions
Quality / Root-Cause Engineer	When a defect is caught late, manually combing through dozens of upstream stations' logs and shift records to find the likely cause	A ranked shortlist of suspect stations the moment a defect is caught, cutting investigation time — while teardown still confirms the true cause
Maintenance / Reliability Lead	Equipment wear is usually caught reactively, forcing unplanned stoppages and rushed repairs	Weeks-to-days of forecasted lead time on developing mechanical issues, schedulable into existing maintenance windows
Leadership / Investment Committee	Digital-twin pitches are common; validated, line-specific evidence that one will actually work is rare	A business case built on the same validated accuracy numbers the floor and plant-manager views rely on — one auditable set of figures, not a separate marketing narrative

4. Prototype Summary

The accompanying repository contains a complete, runnable prototype of the modeling layer described in Section 2 — not a mockup. It simulates a 40-station line (27 rich, 8 basic, 5 manual-tier stations) producing 3,000 units, with three injected, multi-causal fault modes standing in for real equipment wear, an intermittent tool/part-quality interaction, and shift-based operator variation. Every module then runs against that simulated data exactly as it would against a live OT feed, and a validation harness scores every predictive claim against the (normally hidden) ground truth.

4.1 What's included

File	What it does
src/line_config.py	Defines the 40-station topology and sensor tier per station
src/simulate_line.py	Generates the synthetic production event log and the three injected fault modes
src/spc_models.py	Causal EWMA anomaly scoring for all sensed (rich/basic) stations
src/inference_models.py	Shift-based risk inference for the fully manual station, using delayed confirmed attributions
src/bottleneck_forecast.py	Statistically-significant trend forecasting of takt-time breaches

File	What it does
src/defect_traceback.py	Unit-level and cohort-level root-cause ranking for late-surfacing defects
src/validation.py	Precision/recall scoring, threshold sweep, and lead-time measurement against ground truth
src/run_pipeline.py	Orchestrates the full run end-to-end and exports the dashboard's data file
dashboard/index.html	The working three-persona dashboard, self-contained and runnable by opening the file directly

4.2 Validated results from the reference run

These figures come directly from running src/run_pipeline.py against the simulator's default seed — reproducible by anyone cloning the repository, not hand-picked.

Mechanism	Result
Bottleneck forecast lead time (Station 12 case study)	517 units of lead time (~2.1 shifts) between the predictive alert and the sustained takt-time breach
Early-warning defect precision/recall (default threshold)	48.1% precision / 41.9% recall, a ~23x lift over the 2.07% base defect rate — full tradeoff curve tunable from 95.2% precision / 32.3% recall (conservative) to 3.5% precision / 77.4% recall (maximally sensitive)
Root-cause ranking accuracy	69.4% top-3 hit rate against confirmed source station (n=62 confirmed defects); the gap is concentrated at the one fully-manual station with no sensor trail — an explicit, expected limitation, not a modeling failure
Manual-station (zero-sensor) risk inference	Correctly and repeatedly flags the night shift as elevated-risk at the fully manual panel-fit station, using only delayed downstream confirmations — no direct sensor data at all

On the honesty of these numbers

The early-warning precision/recall figures are deliberately reported as a full tradeoff curve, not a single flattering number, because that tradeoff is the actual decision a plant has to make when deploying this system. The root-cause accuracy figure is reported with its known limitation explained, not smoothed over. This is the same standard of evidence Section 5's business case and Section 6's rollout plan are held to.

5. Business Case & Impact

The figures below are illustrative, generated by the prototype from explicitly stated assumptions (not customer data), and scoped to a single pilot line — deliberately conservative rather than optimistic, because an overstated business case is the fastest way to lose credibility with the audience that has to approve it. Every assumption is listed; replace them with site-specific figures before using this for an actual investment decision.

5.1 Pilot-line assumptions and outcome

Assumption	Value	Basis
Annual units (pilot line)	125,000	~2 shifts/day, ~250 units/shift, ~250 production days/year
Simulated escaped-defect rate	2.07%	From the prototype's reference run; replace with the pilot line's actual final-inspection defect rate
Cost per escaped defect (blended)	\$1,200	Rework labor, comeback handling, and minor warranty cost — deliberately not a recall-scale figure
Detection recall used in the case	32%	The high-precision (~95%) operating point from the threshold sweep — i.e., savings are only counted from alerts confident enough to act on, not the maximum-sensitivity setting
Unplanned-stoppage hours avoided/year	16	Roughly two wear-driven failures/year converted from unplanned to scheduled repair via forecasted lead time
Cost per hour of unplanned downtime	\$15,000	Industry-typical single-line stoppage cost; varies significantly by plant and should be replaced with a site figure

Outcome (Year 1, pilot line)	Estimate
Estimated defects avoided / year	~827 units
Estimated annual defect-cost savings	\$992,000
Estimated annual downtime savings	\$240,000
One-time prototype/pilot development cost	\$180,000
One-time per-line rollout cost (integration, retrofit)	\$240,000
Annual platform cost (hosting, monitoring, upkeep)	\$60,000

Outcome (Year 1, pilot line)	Estimate
Estimated Year 1 net	\$752,000
Estimated payback period	~4.7 months

5.2 How this scales beyond one line

The first line carries the full weight of data-engineering and integration cost; every additional line or site reuses the ingestion architecture, the modeling code, and the dashboard views unmodified, per Section 2.8 — the marginal cost of each subsequent rollout is dominated by site-specific configuration and any sensor-tier retrofits that line's own data indicates are worth prioritizing, not by rebuilding the system.

5.3 Soft benefits not captured in the figures above

- Quality engineers spend less time manually trawling upstream logs after a defect is caught, and more time confirming or ruling out a short, ranked list of suspects.
- Maintenance shifts from reactive firefighting to scheduled intervention, improving both cost and safety outcomes.
- Floor supervisors gain visibility into stations they previously had no signal from at all (manual-tier stations), closing a genuine blind spot rather than just making an existing dashboard prettier.
- A validated, auditable accuracy track record becomes the evidence base for future automation and retrofit investment decisions, rather than relying on anecdote.

6. Phased Roadmap

The sequencing below is deliberately conservative: each phase has an explicit exit gate, thresholds start tight (high precision, lower recall) and only loosen as a real accuracy track record accumulates, and every physical change is scheduled into the plant's existing maintenance-window cadence rather than requiring special downtime.

Phase	Duration	Objective & key activities	Exit gate
0. Discovery & data audit	4–6 weeks	Classify every station into the rich/basic/manual taxonomy; map existing OT tags and historian access; identify the next 1–2 scheduled maintenance windows	Signed-off station inventory and integration plan; no production changes yet

Phase	Duration	Objective & key activities	Exit gate
1. Pilot on one line, conservative thresholds	~3 months	Deploy read-only ingestion; run SPC, bottleneck forecasting, and traceback in shadow mode (visible, not yet trusted as the primary source) at a high-precision threshold	Validated precision/recall on real (not simulated) outcomes meets an agreed floor; floor supervisors report alerts as useful, not noisy
2. Calibration & manual-station retrofit	3–6 months	Loosen thresholds as accuracy is proven; install first low-cost retrofits (Section 2.5) at manual stations with the highest confirmed root-cause attribution, timed to the next maintenance window	Root-cause accuracy improves measurably at the retrofitted station(s); manual-station blind spot demonstrably narrowed
3. Multi-line rollout, same plant	6–12 months	Onboard remaining lines using the config-driven approach (Section 2.8); extend the plant-manager view to a cross-line comparison	Two or more lines running with validated accuracy at or above the pilot's benchmark
4. Multi-site rollout	Ongoing	Extend to additional plants using the federated calibration model; leadership view becomes a fleet-level rollout tracker	Consistent validation methodology and reporting across sites, with site-specific calibration

7. Risks & Mitigations

Risk	Impact if unaddressed	Mitigation
False alarms erode floor-level trust	Supervisors start ignoring alerts, defeating the system's purpose	Launch at a high-precision, conservative threshold (Phase 1); run in shadow mode before becoming the primary signal; continuous precision/recall backtesting with an explicit auto-tightening policy if precision drops
Integration risk to live production systems	Any perceived risk to PLCs or line-control logic can halt the project regardless of modeling quality	Strictly read-only data taps with no write path to control logic; all physical installation confined to scheduled maintenance windows; edge buffering so legacy PLCs are never polled harder than they already tolerate

Risk	Impact if unaddressed	Mitigation
Sensor retrofit cost and downtime	Waiting for a full retrofit before delivering value stalls the project indefinitely	The twin delivers value on day one using inference at unsensed stations (Section 2.5); retrofits are incremental, ROI-prioritized by the twin's own output, and timed to existing maintenance windows
Data quality and labeling at manual stations	Inference at unsensed stations depends on the quality and honesty of confirmed downstream attributions	The confirmation-lag design mirrors the real rework/teardown process rather than assuming instant ground truth; operator training and buy-in are part of Phase 1; the system is explicit about lower confidence at manual stations rather than overstating it
Model drift over time or during product changeovers	Thresholds calibrated for one product mix or wear cycle may misfire after a changeover	Continuous validation loop (Section 2.4) monitors drift and recalibrates on a defined cadence; changeovers trigger a scheduled re-baseline rather than silent degradation
Organizational adoption / change management	A technically sound system that nobody on the floor actually uses delivers no value	Role-specific views designed around each persona's actual workflow (Section 2.6); shadow-mode rollout with floor-supervisor champions before the system becomes authoritative
OT-adjacent cybersecurity exposure	New gateways and edge infrastructure expand the attack surface if not properly segmented	Standard OT/IT network segmentation; ingestion layer is read-only and outbound-only from the OT zone; infrastructure subject to the plant's existing security review process before Phase 1 deployment
Multi-site data governance	Inconsistent data handling across sites creates compliance and comparability problems as the rollout scales	Federated calibration architecture (Section 2.8) keeps each site's operational data local while sharing only the modeling framework and aggregated validation metrics

8. Appendix — Repository & Reproducing These Results

Full setup and run instructions are in the repository's README.md. In summary:

- `cd src && python3 run_pipeline.py` — runs the full simulation, all four modeling components, and the validation harness end-to-end (~15 seconds), printing the validation summary and exporting `dashboard/dashboard_data.json`

- Open dashboard/index.html directly in a browser — a self-contained interactive dashboard implementing all three stakeholder views against that run's output
- Individual modules (src/spc_models.py, src/bottleneck_forecast.py, src/inference_models.py, src/defect_traceback.py, src/validation.py) can each be run standalone for inspection

Repository link and demo video are referenced in README.md.